

Workflow hacks for large datasets (in HPC)

ALEX RAZOUMOV
alex.razoumov@westdri.ca



SIMON FRASER
UNIVERSITY



**Digital Research
Alliance** of Canada

Numerical simulations produce too much data!

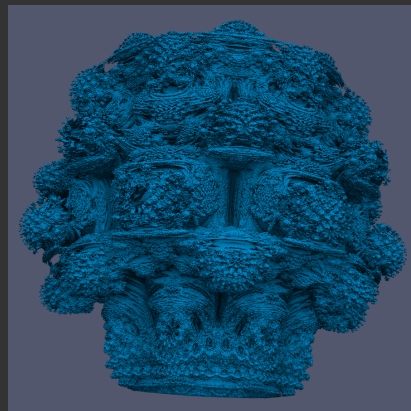
(challenge to write, analyze and store long-term)

- Store every 100th timestep
- Store only selected variables
- Visualize in-situ ⁽²⁾
- Long term: use lossy compression ⁽¹⁾
- Long term: store data in a distributed dataset ⁽¹⁾
- Long term: use modern archival tools ⁽¹⁾

1. <https://training.westdri.ca/tools/rdm>
2. <https://training.westdri.ca/tools/visualization>

Lossless 3D data compression

- Tools like gzip, bzip2, xz and many others
- Use general-purpose algorithms; replace recurrent bit patterns in the data by references to a single copy of the pattern
- Often included into the file format (NetCDF, HDF5, VTK)
- For typical datasets (such as CFD) $\sim 40 - 50\%$ reduction in size
- Very high compression rates when high redundancy is present in the data
 - e.g. the Mandelbulb on the right: 800^3 in single precision $\Rightarrow$ 1.9G, actual NetCDF file 12M
 - effectively, a litmus test for the efficiency of your data storage (if stored uncompressed)



Lossy 3D data compression

- Lower resolution and/or precision
- ✓ Orthogonal transformation-based algorithms, most notably **zfp** (based on local block transforms, similar to discrete cosine transforms)
- ✓ Topological compression
- Compression via ML – probably not any time soon ...

-
- Both **can reliably achieve ~20-30X compression, without any visible data degradation** – and sometimes as high as ~80-200X, making them ideal for storage and archival
 - Work on 1D / 2D / 3D scalar fields, not on files
 - Compression artifacts show in very different ways
 - Can these be applied to other data, e.g. MD trajectories, AMR datasets, etc?

zfp: compressed floating-point and integer arrays

<https://computing.llnl.gov/projects/zfp>

- Developed at LLNL
- Open source <https://github.com/LLNL/zfp>
- Lossless and lossy
- Very good documentation <https://zfp.readthedocs.io>
- Written in C/C++
- Bindings in C, C++, Python (**zfpPy**), Fortran, Julia (**ZfpCompression.jl**)
 - in ParaView via TTK's TopologicalCompressionWriter without topological compression
- Built into ~63 projects, including <https://h5z-zfp.readthedocs.io> and VTK-m
<https://computing.llnl.gov/projects/floating-point-compression/related-projects>
- For faster (de)compression supports several backends: OpenMP, CUDA, HIP (C++ runtime for AMD and NVIDIA GPUs), FPGAs
- Reported throughputs up to 2 GB/s per CPU core, 800 GB/s aggregate throughput on GPUs

Algorithm

- Details at <https://zfp.readthedocs.io/en/release1.0.1/algorithm.html#lossy-compression>
- Relies on orthogonal transforms, keeping only significant transform coefficients
- The transform coefficients are encoded into a losslessly-compressed bit stream that can be truncated in one of three ways, giving a user three “non-expert” compression modes:
 1. **fixed-rate mode** controlled by a double-precision parameter (*rate*) giving a fixed number of bits per floating number
 2. **fixed-precision mode** controlled by an integer parameter (*precision*) specifying the number of bit planes for the transform coefficients
 3. **fixed-accuracy mode** controlled by a double-precision parameter (*tolerance*) specifying the max point-wise variable error

Using zfp via Python: tiny 1D array

```
import numpy as np, zfp
```

```
# first, lossless compression
```

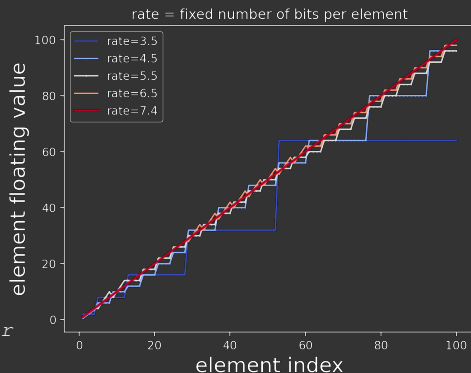
```
initial = np.arange(1, 100+1, dtype=np.float32)
compressed = zfp.compress_numpy(initial)
decompressed = zfp.decompress_numpy(compressed)
np.testing.assert_array_equal(initial, decompressed)
# they are equal
```

```
# next, lossy compression
```

```
rate = 3.5 # number of bits per each floating number
initial = np.arange(1, 11, dtype=np.float32)
compressed = zfp.compress_numpy(initial, rate=rate)
decompressed = zfp.decompress_numpy(compressed)
print("rate_=", rate, "-->", str(len(compressed))+"B", "decompressed_=", decompressed)
```

```
file = open("compressed.zfp", "wb")
file.write(compressed)
file.close()
```

- For this problem, at $r \approx 7.375$ the compressed array becomes **indistinguishable from the original**, probably good enough for somewhat smaller r (depends on your data and your tolerance level)
- ~ 5X compression here; much higher ratio for bigger datasets (next slide)



Using zfp via Python: large 3D array

➡ 3D sine envelope wave function defined inside a unit cube ($x_i \in [0, 1]$)

$$f(x_1, x_2, x_3) = \sum_{i=1}^2 \left[\frac{\sin^2 \left(\sqrt{\xi_{i+1}^2 + \xi_i^2} \right) - 0.5}{\left[0.001(\xi_{i+1}^2 + \xi_i^2) + 1 \right]^2} + 0.5 \right], \text{ where } \xi_i \equiv 15(x_i - 0.5)$$

- Discretized at 300^3 Cartesian grid in double precision `double300.nc` $\Rightarrow$ 206M
- Lossless compression: `double300.nc.gz` $\Rightarrow$ 125M, `double300.nc.bz2` $\Rightarrow$ 119M
- NetCDF's `compression='zlib'` $\Rightarrow$ 159M
- NetCDF's `compression='zlib', significant_digits=3` (very aggressive compression) $\Rightarrow$ 33M

```
import netCDF4 as nc, zfp
```

```
f = nc.Dataset('double300.nc')
rho = f.variables['density'][:]
compressed = zfp.compress_numpy(rho, rate=2.0)
print(len(compressed))    # in bytes => 6.4M
```

```
file = open("compressed.zfp", "wb")
file.write(compressed)
file.close()
```

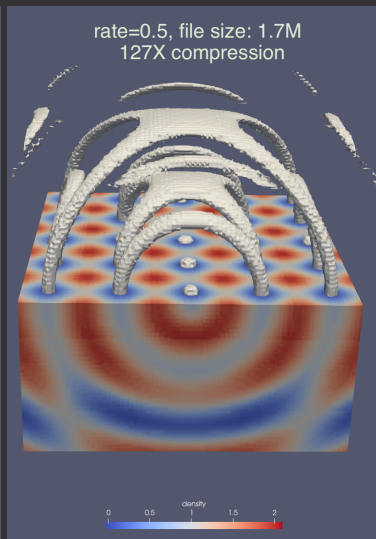
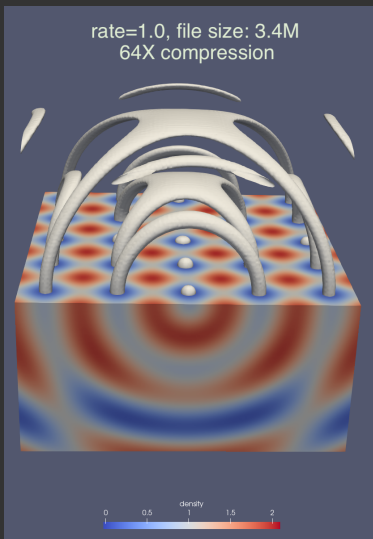
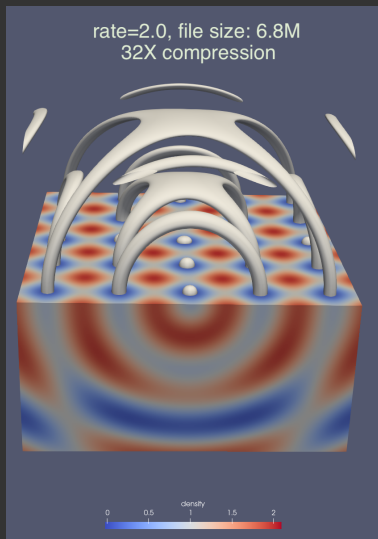
- Demo `rate=2.0, rate=1.0, rate=0.5`

```
import netCDF4 as nc, zfp
```

```
file = open("compressed.zfp", "rb")
compressed = file.read()
file.close()
decompressed = zfp.decompress_numpy(compressed)
```

```
f = nc.Dataset('compressed.nc', 'w', format='NETCDF4')
nx, ny, nz = decompressed.shape
f.createDimension('x', nx)
f.createDimension('y', ny)
f.createDimension('z', nz)
rho = f.createVariable('density', 'f8', ('x','y','z'))
rho[:, :, :] = decompressed
f.close()
```


Using zfp via Python: large 3D array (cont.)

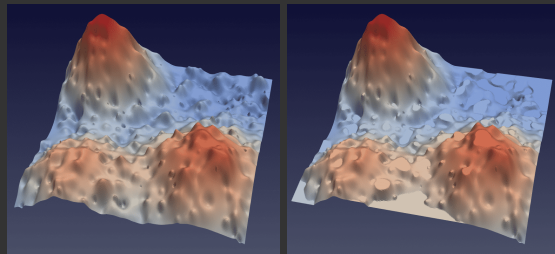


Topological compression

- Implemented in TTK = the Topology ToolKit <https://topology-tool-kit.github.io>
- Interfaces:
 - pure C++
 - VTK/C++ (~3X shorter code)
 - ParaView Python (~5X shorter code) and ParaView GUI
- Noticeably slower than **zfp**

Algorithm:

1. Topological simplification: remove all topological features below some **persistence tolerance ϵ**
2. Domain quantization
3. Approximate data in each domain with an adaptive step function: is max pointwise error larger than the **user-specified error** $\Rightarrow$ subdivide that region in two
4. Lossless compression of all elements



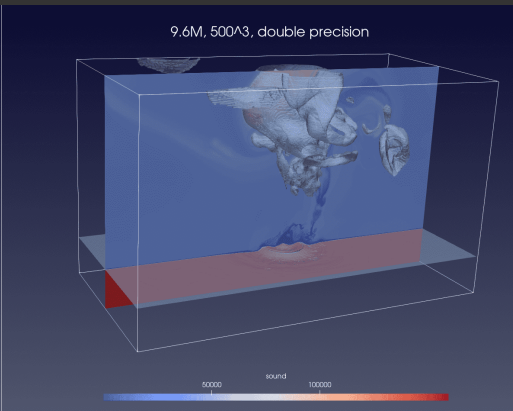
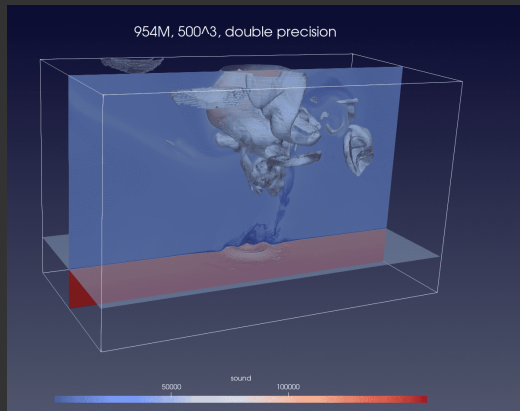
Deep water impact dataset

IEEE Vis 2018 contest <https://sciviscontest2018.org>

500 ³ uncompressed	gzip	LZMA	ZLib
954M	245M	178M	258M

- Original: double-precision, single-variable VTK file (snd.vti)
- Compressing with C++/VTK:

```
../lossy/main -i snd.vti -t 10 -e 10 -o t10e10.ttk
```
- ~1 min to compress and ~1 min to uncompress (or load into ParaView)



What's in-situ?

- Modern parallel simulations can generate huge amounts of data → I/O bottleneck
 - large HPC codes cannot afford to write full data to disk every time step

What's in-situ?

- Modern parallel simulations can generate huge amounts of data → I/O bottleneck
 - large HPC codes cannot afford to write full data to disk every time step
- **In-situ visualization** idea: instead of writing full 3D variables to disk, only write either:
 1. some derived data needed for visualization (e.g. 2D polygonal isosurfaces), or
 2. images directly from the simulation code
 - much smaller disk output, no need to write full 3D datasets every time step
 - sometimes referred to as “**co-processing**”

What's in-situ?

- Modern parallel simulations can generate huge amounts of data → I/O bottleneck
 - large HPC codes cannot afford to write full data to disk every time step
- **In-situ visualization** idea: instead of writing full 3D variables to disk, only write either:
 1. some derived data needed for visualization (e.g. 2D polygonal isosurfaces), or
 2. images directly from the simulation code
 - much smaller disk output, no need to write full 3D datasets every time step
 - sometimes referred to as “**co-processing**”
- Wait ... writing images or smaller datasets to disk directly from simulation codes is nothing new: we've been doing it for the last 50 years

What's in-situ?

- Modern parallel simulations can generate huge amounts of data → I/O bottleneck
 - large HPC codes cannot afford to write full data to disk every time step
- **In-situ visualization** idea: instead of writing full 3D variables to disk, only write either:
 1. some derived data needed for visualization (e.g. 2D polygonal isosurfaces), or
 2. images directly from the simulation code

→ much smaller disk output, no need to write full 3D datasets every time step

 - sometimes referred to as “**co-processing**”
- Wait ... writing images or smaller datasets to disk directly from simulation codes is nothing new: we've been doing it for the last 50 years
- Let's rephrase: with ParaView Catalyst / VisIt LibSim, **in-situ** refers to exposing your runtime in-memory simulation data arrays to the visualization library (without writing any data to disk) while using familiar interactive Python pipelines to process and visualize data
 1. instrument your simulation code once
 - Catalyst is extremely diligent about not duplicating any data arrays in memory, passing only pointers and data description via its API ⇒ some limitations on simulation data structures
 2. modify and apply your analysis/visualization pipelines without recompiling your code

Catalyst evolution

- Circa 2012-2013 - development of the original Catalyst library
 - relied heavily on users mapping their simulation code's data structures to VTK data objects; required good knowledge of the VTK data model
 - it was relatively easy to make mistakes duplicating data arrays in memory, as there are different ways to initialize VTK data APIs depending on the data object type
 - ParaView / VTK codebase is continually changing → simulation code maintainers had to continuously update their Catalyst adaptor, or even maintain multiple code builds to work with different ParaView versions

Catalyst evolution

- Circa 2012-2013 - development of the original Catalyst library
 - relied heavily on users mapping their simulation code's data structures to VTK data objects; required good knowledge of the VTK data model
 - it was relatively easy to make mistakes duplicating data arrays in memory, as there are different ways to initialize VTK data APIs depending on the data object type
 - ParaView / VTK codebase is continually changing → simulation code maintainers had to continuously update their Catalyst adaptor, or even maintain multiple code builds to work with different ParaView versions
- 2020 - first release of Catalyst2
 - for user-provided simulation data (meshes and fields) description switched to Conduit API <https://llnl-conduit.readthedocs.io>
 - converting of a Conduit mesh description to an appropriate VTK data object is done inside the Catalyst2 library (not exposed to user) → the Catalyst adaptor inside the simulation code should be relatively stable across many versions of ParaView / VTK

Core Catalyst API: converting data structures to VTK, few functions to initialize / update / finalize the in-situ analysis (standalone GitHub repo)

Combined ParaView-Catalyst API: the ParaView-based implementation of the Catalyst API = core Catalyst API library + collection of Catalyst-related libraries from a specific ParaView build

Getting started with Catalyst2 API

1. Download and compile core Catalyst2 library from `https://gitlab.kitware.com/paraview/catalyst.git`
- 2a. Compile ParaView server from source `https://www.paraview.org/download`
 - use `PARAVIEW_ENABLE_CATALYST=ON`
 - at runtime you'll need quite a few Catalyst-related libraries from a ParaView build of the same version you will use to create Catalyst scripts
- 2b. Alternatively, you could use precompiled (downloaded) ParaView, however there are numerous online reports of problems that only go away when you compile ParaView from source; likely related to the included MPI version
3. In ParaView's source code, start with the examples `ParaView-v5.12.1/Examples/Catalyst2`
4. Further examples in `standalone-mpi/{imageData,unstructuredData}` (two versions of the same code), both included in the webinar's ZIP file
 - compile and then run either serial or parallel code

```
cd standalone-mpi/unstructuredData && make && ./bin/CFullExampleV2 <flags_and_parameters>
mpirun -np <number_of_cores> ./bin/CFullExampleV2 <flags_and_parameters>
```

Modifying the code (intentionally kept all the original file and function names)

<code>FEDriver.c</code>	the main loop of the simulation
<code>FEDataStructures.c</code>	define simulation data in <code>InitializeGrid()</code> , <code>InitializeAttributes()</code> , <code>UpdateFields()</code>
<code>CatalystAdaptor.h</code>	create a Conduit node to describe our data and pass it to the Catalyst API
<code>catalyst_pipeline.py</code>	Catalyst script called at runtime

- Catalyst API provides 3 main functions to pass data and control from a simulation code to the ParaView-Catalyst implementation: **`catalyst_initialize()`**, **`catalyst_execute()`**, and **`catalyst_finalize()`**
- Each of the three functions is passed a **Conduit Node** object
 - think of it as a light-weight container with hierarchical construction (nodes inside nodes)
 - a top-level node **`catalyst_load`** loads a specific Catalyst implementation, e.g. ParaView-Catalyst
 - if PV-Catalyst, it looks for a top-level node named **`catalyst`**
 - **`catalyst_initialize()`** processes **`catalyst/scripts/name`** (Python script for in-situ analysis), **`catalyst/pipelines`** (hierarchy of various pipeline parameters)
 - **`catalyst_execute()`** processes **`catalyst/state/timestep`**, **`catalyst/state/time`**, **`catalyst/channels/<channel-name>`**, **`channels/<channel-name>/type`**, **`channels/<channel-name>/data`**, and further hierarchy to describe the mesh topology, point coordinates, maybe cell connectivity, and fields
 - **`catalyst_finalize()`** takes an empty node

The default ParaView Catalyst script

Possible command at runtime:

```
mpirun -np 2 ./bin/CFullExampleV2 catalyst_pipeline.py    # each MPI rank will print
```

- catalyst_pipeline.py that came with the example does not do much ...

```
from paraview.simple import *

print("executing catalyst_pipeline")

# registrationName must match the channel name used in Catalyst Adaptor
producer = TrivialProducer(registrationName="grid")

# this function is triggered when there is new data in the channel
def catalyst_execute(info):
    global producer
    print("-----")
    print("executing (cycle={}, time={})".format(info.cycle, info.time))
    producer.UpdatePipeline()
    print("bounds:", producer.GetDataInformation().GetBounds())
    print("velocity-magnitude-range:", producer.PointData["velocity"].GetRange(-1))
    print("density-range:", producer.CellData["density"].GetRange(0))
```

Representative dataset

- We want to replace `catalyst_pipeline.py` with our own **Python visualization script** processing our data in some way and then writing output with an image/data Extractor
 1. load a **representative dataset** and process it in the GUI
 2. apply an Extractor (image or data)
 3. File | Save Catalyst State to save your script
- Our representative dataset must:
 1. be of the same dataset type, e.g. `vtkUnstructuredGrid`, `vtkImageData`, etc.
 2. have the same attributes defined over the grids as the simulation adaptor code will provide to Catalyst during simulation runs
 - + to generate the script in the GUI, we must use the same ParaView version as the version of ParaView Catalyst that the simulation code will run with
- Where do we get a representative dataset?
 - can create it from scratch via a combination of Sources (for grids) and Calculators (for 3D variables), perhaps using a Programmable Filter for Unstructured Grids – straightforward but slow manual process
 - in Catalyst2 a generic `gridwriter.py` was under development – not sure of its status ...
 - can add a data writer to your Catalyst Adaptor following the C++ example in `ParaView-v5.12.1/Examples/Catalyst2/CxxImageDataExample`

Representative dataset → Catalyst script → in-situ visualization

- Adjust the number of time steps, recompile and then run

```
./bin/CFullExampleV2 --output dataset-%04ts.vtpd
```

- Catalyst always produces a partitioned dataset
- if run in parallel, would produce multiple files per timestep

1. Load one of the timesteps into ParaView
2. Apply filters to create a visualization you like
3. Apply Extractors | Image | PNG (writing images) or Extractors | Data | VTP (writing data)
 - these create various Catalyst functions that get triggered when you run `catalyst().execute` from your simulation code (typically, once per timestep)
4. File | Save Catalyst State to save it as `extract-image.py`
5. Make sure `registrationName` in the script matches the data channel name in the simulation code
 - alternatively, could do this by renaming the first data object in the pipeline browser
6. Run

```
./bin/CFullExampleV2 extract-image.py
```

7. Your PNG images or VTP data will be in `datasets/` directory
 - if writing data, check the file sizes of the original 3D and the derived 2D data

ParaView Live

- Idea: load data from your live simulation on the cluster into a ParaView server (without writing this data to disk!) and view it in a ParaView client on your computer
-
- For simplicity assuming that both the ParaView server + client are running on your computer as part of the ParaView application
 - In principle, it *might be tempting* to set up a ParaView server (talking to the simulation) on a remote machine with a higher bandwidth to the cluster, and the ParaView client on your computer, but it is actually not a good idea, since **all heavy processing should be done in-situ by the simulation code**, and you should be sending **much smaller, derived datasets** to the ParaView server
 - Our ParaView application acts as a **server (with a GUI on top) that your live simulation on the cluster is connecting to** (avoiding the term “client” on purpose)
 - This is not for production visualization, but rather for interactive work, e.g.
 - creating/debugging Catalyst scripts, or
 - debugging your simulation
 - Ideally, all production visualization should be done non-interactively via Catalyst scripts

In-situ summary

- Steep learning curve ... however, once implemented in a simulation code, you do not even need to recompile the code to switch to a new pipeline
- Catalyst Python pipelines can be created interactively in the GUI
- Catalyst can produce both images and derived ($\Rightarrow$ smaller) data
 - writes partitioned data, one file per MPI rank
- Can be used from a variety of languages: C, C++, Fortran, Python, Julia
- Catalyst has been shown to scale to millions of CPU cores
- Design philosophy: use Conduit nodes to pass pointers to existing arrays in memory
 - arrays will not be duplicated
 - you need to describe the mesh type / coordinates / topology
 - do not need to know the underlying VTK data model
 - internally, supports all VTK data types: ImageData, StructuredGrid, UnstructuredGrid, ...
- Make sure to use latest Catalyst2 (a significant rewrite of the original library)
- Be careful when enabling data in ParaView Live
- To dive deeper, check out <https://sensei-insitu.org> – data producers talking to generic in-situ endpoints via a unified API, can combine multiple endpoints in a single simulation

DIY distributed storage of large datasets

- Git was designed for version control of text files (codes, articles, any text-based documents)
- Putting large data files under Git will double the size of your repository (wasted space)
- Poor man's solution: put the repository (`.git/`) on a drive separate from the working directory
 - ✓ two copies on two separate drives (1 copy per drive)
 - ✗ if you modify/delete a large binary file, a copy still in the repository wasting space
- Multiple “proper” solutions: git-annex, Git LFS, DataLad, DVC – we gave webinars on most of these
 - DataLad and DVC have additional features to track data origin, provision input and control output when running pipelines (not the primary focus here); I find them a little too prescriptive
 - git-annex and Git LFS are simpler, designed specifically to help Git manage large files
 - git-annex uses symlinks, fully distributed, can mix multiple backends (local, external drives, scp, cloud), excellent offline support, no native GitHub/GitLab
 - Git LFS uses pointer files, relies on a centralized server or remote, limited offline support, GitHub/GitLab support

DIY distributed storage of large datasets (cont.)

- **Specific use case requirements:**

- single collection spanning multiple external drives and remote (ssh) filesystems
 - ability to keep everything organized (arbitrary folder hierarchy) and in sync
 - searchable even when offline
 - at least a single backup copy of everything, multiple+remote backups for more important files
- ⇒ much prefer git-annex for this due to its fully distributed nature

- **Key idea:** file organization is orthogonal to data distribution across mutiple locations

- **Key idea:** each annex (repository) has full knowledge of all files (but not their content) and of their locations

Initialize an annex

```
alias ga='git-annex'

mkdir earth mars venus    # placeholders for different drives or filesystems
mkdir earth/annex && cd earth/annex
git init
ga init "earth"

function populate() {
    # takes a number N, creates N large random binary files
    ...
}
populate 10

ga add .    # 1. move files to .git/annex/objects
            # 2. create symbolic links pointing to the new location
            # 3. add these links to the index (git staging area)
ga sync    # 1. run local 'git commit' with a generic message
            # 2. update the current repository with all changes to its remotes (if any)
            # 3. push any changes in the current repository to its remotes (if any)

ga status
ls
ga whereis  # show annexed files and their locations
```

Undo accidental add command

```
populate 1
ls
ga add test86593
ga status          # we have annexed the file, added its link to the staging area
ls -l test86593    # the file has been annexed
ls -l $(readlink test86593) # its content is stored in this object

ga unannex test86593 # (1) put the file content back to replace the link
ga unused           # show all unlinked files in the object store
ga dropunused --force 1 # (2) drop the first unused object
ga unused           # no unlinked files left
```

Remove a file from annex that was both added and committed – the above steps, plus:

```
ga sync          # run local 'git commit' with a generic message
ga status        # now we have a new, untracked file
```

How do you remove multiple files or a directory?

```
ga unannex <file1> <file2> <file3>
ga unannex <subdir>
ga unused           # show all unlinked objects
ga dropunused --force {1..5} # adjust the number based on the previous command's output
```

Alternative way to remove a file completely

```
ga drop --force <filename> # drop a local copy (object)
unlink <filename>          # remove the link
ga sync                   # does git commit in the background
```

Remote via a clone

```
ga status      # make sure everything is committed
cd ../../venus
git clone ../../earth/annex
cd annex
ls             # there are symbolic links to non-existent objects
du -s .       # currently no objects (data files)
git log        # see the history from earth
ga whereis     # all 10 files are on earth (1 copy of each)

populate 10
ga add .
git commit -m "add_10_new_files" # as an alternative to `ga sync` in this context

ga whereis     # 10 files on earth, 10 files here

ga sync        # sync to earth
cd ../../earth/annex/
ga sync        # sync from venus
ga whereis     # can see all 20 files now (10 local, 10 on venus)
```

Can also create a remote from scratch (an isolated annex), add the original repo as a remote, and merge their “unrelated” histories

Get multiple copies

```
ga sync
ga get --auto --numcopies=2 <file1> <file2> ...    # get files with fewer than 2 copies
ga whereis <file1> <file2> ...
ga sync
```

Other operations

```
ga find                # list all local files
ga drop --auto --numcopies=<N> <path>             # drop a local copy if there are N remote copies
ga copy <filename> --to <annexName>               # copy a file to a remote annex
ga copy <filename> --from <annex>                 # copy a file from a remote annex
ga move <items> --to here
ga move <items> --to <annex>
ga info <path>        # show local+global disk usage in a directory or per file
```

Annex summary

- Cognitive clash between `git-annex <verb>` and `git <verb>` commands
 - ⇒ replace `git-annex` with `ga` (or something sufficiently distinct)
 - write `ga()` function to add extra functionality
- Easy to extend to SSH remotes, but sync'ing across them might be slow
 - ⇒ turn them on only when needed, e.g.

```
git remote add cluster <username>@<cluster>.alliancecan.ca:/path/to/your/project/annex
...
git remote remove cluster
```

TAR → DAR (Disk ARchiver)

- To avoid problems on parallel file systems, we always advise researchers to pack their multiple small files into TAR archives
- TAR has its limitations:
 - introduced to Unix in January 1979 for sequential write/read on tape drives
 - ⇒ **no index for random access** to TAR contents
 - ⇒ searching and extracting could be very slow
 - ⇒ some third-party tools add indexing
 - padding for each file (to reach a multiple of 512 bytes) and at the end of the TAR file
 - some other limitations

TAR → DAR (Disk ARchiver) (cont.)

- DAR was written from the ground up as a modern replacement to TAR
- Open-source, first released in 2002, still actively maintained
 - full / differential / incremental backup
 - each archive includes an index ⇒ fast file list/restore
 - build-in compression on a file-by-file basis: more resilient against data corruption, can avoid compressing already compressed files, e.g. `-Z "*.mp4"` `-Z "*.gz"`
 - strong encryption
 - can split archives at 1-byte resolution
 - supports extended file attributes, sparse files, hard and symbolic/soft links
 - can detect corruption in both headers and saved data, recover with minimal data loss
 - can merge two archives into a new one, e.g. can convert differential/incremental backup to full
- Full DAR /TAR comparison <http://dar.linux.free.fr/doc/FAQ.html#tar>

Seeking time

- Don't benchmark this on a cluster!
- On a laptop quickly generate 17GB of random data; $\$RANDOM \in [0, 32767 = 2^{15} - 1]$

```
cd /Volumes/boa/tmp    # on an external HDD (slower for a purpose)
mkdir test
for num in $(seq -w 000 999); do    # emulates inefficient cluster workflow
    echo $num
    # generate a binary file (1-33)MB in size
    dd if=/dev/urandom of=test/test"$num" bs=1024 count=$(( RANDOM + 1024 ))
done
```

- Create TAR and DAR archives

```
time tar cvf all.tar test          # 5m57.084s; created all.tar
time dar -w -c all -g test         # 6m27.853s; created all.1.dar
```

- Extract a single file: to clear cache, before each command delete test/test596, unmount the drive, mount it again, cd into parent directory

```
time tar xvf all.tar test/test596          # 7.876s   7.884s   7.785s
time dar -O -w -x all -v -g test/test596    # 0.525s   0.662s   0.594s
# dar -O -w -x all -v -g test    # to restore everything
```

Create a basic DAR archive

```
mkdir test
for num in $(seq -w 00 99); do
    echo $num; dd if=/dev/urandom of=test/test"$num" \
        bs=8 count=$(( RANDOM + 1024 ))
done
dar -w -c monday -g test      # create monday.1.dar
dar -l monday                # list its contents
mkdir restore
dar -R restore/ -O -w -x monday -v -g test/test59    # extract one file
dar -R restore/ -O -w -x monday -v -g test           # extract entire directory
```

monday.1.dar

base name

slice name

file/backup name

Create first incremental backup tuesday.1.dar

```
dd if=/dev/urandom of=test/tue1 bs=8 count=$(( RANDOM + 1024 ))    # add another random file
dar -w -A monday -c tuesday -g test
```

Create second incremental backup wednesday.1.dar

```
dd if=/dev/urandom of=test/wed1 bs=8 count=$(( RANDOM + 1024 ))    # add another random file
dd if=/dev/urandom of=test/wed2 bs=8 count=$(( RANDOM + 1024 ))    # add another random file
/bin/rm test/test99
dar -w -A tuesday -c wednesday -g test
```

Check each backup for test/test99

```
dar -l monday | grep test99      # it is there
dar -l tuesday | grep test99     # it is there
dar -l wednesday | grep test99   # shows REMOVED ENTRY
```

Restore latest backup

```
dar -R restore -O -x wednesday   # will restore only the last incremental backup (wed1, wed2)
```

To restore everything ending with the latest backup, start with the full backup and go sequentially through all incremental backups

```
dar -R restore -O -w -x monday    # restore the first full backup
dar -R restore -O -w -x tuesday   # restore tuel
dar -R restore -O -w -x wednesday # restore wed1, wed2; delete test99
```

- Easy to limit the size of each slice
- Easy to limit the number of files per each slice (with some scripting)
- Encryption built in

DAR summary

- DAR is a modern-day command-line replacement to TAR with many nice features
- Exercise common sense, don't use DAR as a black box:
 - when archiving, monitoring DAR file sizes really helps
 - don't delete any file that you want to keep until you are completely sure it is in the archive
 - we assume that you are below your quota (so you can write files), have read and write permissions, etc.

Questions?